FirstOffer

DATA SCIENCE / ML INTERVIEW KIT

20 Machine Learning Questions for Freshers

The ML fundamentals interviewers check in data science, ML engineer and analyst rounds — overfitting, metrics, trees, scaling, clustering and the end-to-end workflow — explained simply with runnable scikit-learn code.

ML basics | Evaluation metrics | Algorithms | Feature engineering | scikit-learn

20 questions

18 tested examples

5 topics

firstoffer.online

Fresher jobs · Direct apply links · No middlemen

Written from the interviewer's side · Easy → Medium

START HERE

How to use this kit

1
Read the question and answer it out loud before you look at the model answer.

2
Read “What the interviewer is checking” — that's what you're actually scored on.

3
Run the code yourself, change one thing, and predict the new output before running it again.

SPOT THE CLUE

If the question says… reach for…

“Predict a number”	**Regression → MAE · RMSE · R^2**
“Predict a category”	**Classification → precision · recall · F1 · ROC-AUC**
“Find groups, no labels”	**Clustering → K-means**
“Great on train, bad on test”	**Overfitting → simpler model, more data, regularisation**
“Only 2% positives”	**Imbalanced → don't trust accuracy; class weights, PR curve**
“Features on different scales”	**StandardScaler (fit on train only)**
“Text categories as input”	**One-hot encoding**
“How reliable is my score?”	**Cross-validation**

IN THE INTERVIEW ROOM

1 · Define the problem & metric **2 · Explore the data** **3 · Baseline model**
4 · Improve features/model **5 · Validate honestly** **6 · Explain the result**

ABOUT THE CODE

Code uses scikit-learn's built-in datasets (no downloads), with fixed `random_state` so you get the same numbers.

CONTENTS

20 questions, easy to medium

ML fundamentals

01	What are supervised, unsupervised and reinforcement learning?	Easy
02	Classification vs regression — how do you decide?	Easy
03	Why do we split into train and test sets? What is data leakage?	Easy
04	What is overfitting vs underfitting? How do you spot and fix it?	Easy
05	What is k-fold cross-validation and why use it?	Easy

Evaluating models

06	Why can accuracy be misleading? Explain the confusion matrix.	Easy
07	Precision vs recall — when do you care more about each?	Medium
08	What is ROC-AUC?	Medium
09	MAE vs RMSE vs R^2 for regression?	Easy

Core algorithms

10	How does linear regression work?	Easy
11	Logistic regression is used for classification — why is it called regression?	Easy
12	How does a decision tree decide where to split?	Medium
13	Random forest vs gradient boosting? Bagging vs boosting?	Medium
14	How does K-means work, and how do you choose K?	Medium

Features & data preparation

15	When do you need feature scaling? Standardisation vs normalisation?	Easy
16	How do you encode categorical features?	Easy
17	How do you handle an imbalanced dataset?	Medium
18	What is regularisation? L1 vs L2?	Medium

Putting it together

19	How do you explain which features matter to a model?	Medium
20	Walk me through an ML project end to end.	Medium

ML fundamentals

01 What are supervised, unsupervised and reinforcement learning?

Basics Easy

> **WHAT THE INTERVIEWER IS CHECKING**
> Clear definitions with one real example each.

MODEL ANSWER

- Supervised: learn from labelled examples (input → known answer). E.g. predict whether a loan defaults.
- Unsupervised: no labels; find structure. E.g. group customers into segments.
- Reinforcement: an agent learns by trial and error from rewards. E.g. a game-playing bot.
- Bonus: semi-supervised / self-supervised learning uses lots of unlabelled data — that's how LLMs are pre-trained.

Common mistake Calling clustering 'classification' — classification needs labels.

Interviewer may ask next Which type is a spam filter? (Supervised classification.)

02 Classification vs regression — how do you decide?

Basics Easy

> **WHAT THE INTERVIEWER IS CHECKING**
> Mapping a business question to the right ML task.

MODEL ANSWER

- Look at the target: a category (yes/no, spam/not spam, A/B/C) → classification.
- A continuous number (price, salary, days to deliver) → regression.
- The metric changes too: accuracy/F1 for classification, MAE/RMSE for regression.

Common mistake Treating 'rating 1–5' blindly — it can be framed either way; say which and why.

Interviewer may ask next 'Will this customer churn next month?' — which one? (Classification; the probability output is useful too.)

03 Why do we split into train and test sets? What is data leakage?

Basics Easy

WHAT THE INTERVIEWER IS CHECKING

Understanding honest evaluation — very important in real projects.

MODEL ANSWER

- The model must be judged on data it has never seen; the test set simulates future data.
- Leakage: information from the test set (or from the future) sneaks into training, so scores look great but fail in production.
- Classic leaks: scaling/encoding fitted on the full data, a feature created after the outcome, duplicates across train and test.
- For time-based data, split by time, not randomly.

PYTHON

```
from sklearn.datasets import load_breast_cancer
X, y = load_breast_cancer(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)
print(X_train.shape, X_test.shape)
print("positive rate train/test:", round(y_train.mean(), 3), round(y_test.mean(), 3))
```

OUTPUT ✓ REAL RUN

```
(455, 30) (114, 30)
positive rate train/test: 0.626 0.632
```

Common mistake Calling fit_transform on the whole dataset before splitting.

Interviewer may ask next What does stratify=y do? (Keeps the class ratio the same in train and test.)

04 What is overfitting vs underfitting? How do you spot and fix it?

Bias–variance | Easy

WHAT THE INTERVIEWER IS CHECKING

The single most important ML concept for freshers.

MODEL ANSWER

- Overfitting: the model memorises training data (high train score, much lower test score) — high variance.
- Underfitting: too simple to capture the pattern (low on both) — high bias.
- Fix overfitting: simpler model, limit depth, regularisation, more data, cross-validation, early stopping.
- Fix underfitting: more features, a more flexible model, less regularisation.

PYTHON

```
from sklearn.datasets import load_breast_cancer
from sklearn.tree import DecisionTreeClassifier
X, y = load_breast_cancer(return_X_y=True)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.3, random_state=0)
for depth in [1, 3, None]:
    m = DecisionTreeClassifier(max_depth=depth, random_state=0).fit(X_tr, y_tr)
    print(f"depth={str(depth):4}  train={m.score(X_tr, y_tr):.3f}  test={m.score(X_te,
y_te):.3f}")
```

OUTPUT ✓ REAL RUN

```
depth=1     train=0.930  test=0.895
depth=3     train=0.967  test=0.947
depth=None  train=1.000  test=0.912
```

Common mistake Only reporting the training score.

Interviewer may ask next Explain the bias–variance trade-off in one sentence.

05 What is k-fold cross-validation and why use it?

Validation Easy

WHAT THE INTERVIEWER IS CHECKING

Knowing a single split can be lucky or unlucky.

MODEL ANSWER

- Split the training data into k parts; train on k−1 and validate on the remaining one; repeat k times.
- Report the mean and spread of the k scores — a much more reliable estimate than one split.
- Use it for comparing models and tuning hyperparameters; keep a final test set untouched.

PYTHON

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import cross_val_score
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
X, y = load_breast_cancer(return_X_y=True)
model = make_pipeline(StandardScaler(), LogisticRegression(max_iter=1000))
scores = cross_val_score(model, X, y, cv=5)
print(scores.round(3), "mean:", scores.mean().round(3))
```

OUTPUT ✓ REAL RUN

```
[0.982 0.982 0.974 0.974 0.991] mean: 0.981
```

Common mistake Tuning hyperparameters on the test set — then it isn't a test set any more.

Interviewer may ask next What is GridSearchCV? (Tries every parameter combination using cross-validation.)

Evaluating models

06 Why can accuracy be misleading? Explain the confusion matrix.

Metrics Easy

WHAT THE INTERVIEWER IS CHECKING

Thinking beyond accuracy on imbalanced data.

MODEL ANSWER

- If 98% of transactions are genuine, a model that always says 'genuine' is 98% accurate — and useless.
- The confusion matrix splits predictions into TP, FP, FN, TN so you see each kind of mistake.
- Precision = TP / (TP + FP). Recall = TP / (TP + FN). F1 = their harmonic mean.

PYTHON

```
from sklearn.metrics import confusion_matrix, accuracy_score, recall_score
y_true = np.array([0] * 95 + [1] * 5)          # 5% fraud
y_lazy = np.zeros(100, dtype=int)                 # always 'not fraud'
print("accuracy:", accuracy_score(y_true, y_lazy))
print("recall  :", recall_score(y_true, y_lazy))
print(confusion_matrix(y_true, y_lazy))           # rows = actual, cols = predicted
```

OUTPUT ✓ REAL RUN

```
accuracy: 0.95
recall  : 0.0
[[95  0]
 [ 5  0]]
```

Common mistake Celebrating 95% accuracy without checking the class balance.

Interviewer may ask next Which cell of the matrix is a 'false negative'?

07 Precision vs recall — when do you care more about each?

Metrics Medium

WHAT THE INTERVIEWER IS CHECKING

Linking the metric to the business cost of mistakes.

MODEL ANSWER

- Precision: of everything I flagged, how much was right? Care when false alarms are costly (spam filter hiding real emails).
- Recall: of all real positives, how many did I catch? Care when misses are costly (disease screening, fraud).
- They trade off: lowering the decision threshold raises recall and usually lowers precision.

PYTHON

```
from sklearn.datasets import load_breast_cancer
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import precision_score, recall_score
X, y = load_breast_cancer(return_X_y=True)
y = 1 - y                                   # make 1 = malignant (the case we must catch)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.3, random_state=1, stratify=y)
m = make_pipeline(StandardScaler(), LogisticRegression(max_iter=1000)).fit(X_tr, y_tr)
proba = m.predict_proba(X_te)[:, 1]
for t in (0.5, 0.2):
    pred = (proba >= t).astype(int)
    print(f"threshold {t}: precision={precision_score(y_te, pred):.3f} recall=
{recall_score(y_te, pred):.3f}")
```

OUTPUT ✓ REAL RUN

```
threshold 0.5: precision=0.983 recall=0.906
threshold 0.2: precision=0.923 recall=0.938
```

Common mistake Assuming the 0.5 threshold is fixed — it's a business decision.

Interviewer may ask next When is F1 a good single metric? (When you need a balance and positives are rare.)

08 What is ROC-AUC?

Metrics Medium

WHAT THE INTERVIEWER IS CHECKING

An intuitive meaning, not just 'area under the curve'.

MODEL ANSWER

- The ROC curve plots true positive rate vs false positive rate across all thresholds.
- AUC = the chance the model ranks a random positive above a random negative. 0.5 = random guessing, 1.0 = perfect ranking.
- It's threshold-independent; for very imbalanced data the precision–recall curve is often more informative.

PYTHON

```
from sklearn.metrics import roc_auc_score
y_true = [0, 0, 1, 1, 0, 1]
scores = [0.1, 0.4, 0.35, 0.8, 0.3, 0.9]   # model probabilities
print("AUC:", round(roc_auc_score(y_true, scores), 3))
```

OUTPUT ✓ REAL RUN

```
AUC: 0.889
```

Common mistake Saying AUC 0.8 means '80% accurate'.

Interviewer may ask next Of the 9 positive–negative pairs here, how many are ranked correctly? (8 → AUC 8/9.)

09 MAE vs RMSE vs R^2 for regression?

Metrics Easy

WHAT THE INTERVIEWER IS CHECKING

Picking an error metric you can explain to stakeholders.

MODEL ANSWER

- MAE: average absolute error, in the target's units — easiest to explain ('off by ₹2,000 on average').
- RMSE: squares errors first, so big misses are punished more.
- R^2: share of variance explained (1 = perfect, 0 = no better than predicting the mean, can be negative).

PYTHON

```
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
actual    = np.array([30, 45, 50, 60, 80])     # delivery time, minutes
predicted = np.array([32, 40, 52, 58, 95])
print("MAE :", mean_absolute_error(actual, predicted))
print("RMSE:", round(np.sqrt(mean_squared_error(actual, predicted)), 2))
print("R²  :", round(r2_score(actual, predicted), 3))
```

OUTPUT ✓ REAL RUN

```
MAE : 5.2
RMSE: 7.24
R²  : 0.81
```

Common mistake Comparing RMSE across datasets with different units or scales.

Interviewer may ask next Why is RMSE larger than MAE here? (The 15-minute miss is squared.)

10 How does linear regression work?

Algorithms Easy

WHAT THE INTERVIEWER IS CHECKING

The idea of fitting a line by minimising squared error.

MODEL ANSWER

- It fits y = b0 + b1·x1 + ... + bn·xn, choosing coefficients that minimise the sum of squared errors.
- Each coefficient = change in y for a one-unit change in that feature, other features held constant.
- Assumptions: roughly linear relationship, independent errors, constant error variance, little multicollinearity.

PYTHON

```
from sklearn.linear_model import LinearRegression
exp_years = np.array([[0], [1], [2], [3], [5]])
salary    = np.array([4.0, 5.1, 6.0, 7.2, 9.0])     # LPA
lr = LinearRegression().fit(exp_years, salary)
print("intercept:", round(lr.intercept_, 2), " slope:", round(lr.coef_[0], 2))
print("predict 4 yrs:", round(lr.predict([[4]])[0], 2))
```

OUTPUT ✓ REAL RUN

```
intercept: 4.05  slope: 1.0
predict 4 yrs: 8.06
```

Common mistake Reading a coefficient as causal — it's an association in this data.

Interviewer may ask next What is multicollinearity and why is it a problem? (Correlated features make coefficients unstable.)

11 Logistic regression is used for classification — why is it called regression?

Algorithms Easy

WHAT THE INTERVIEWER IS CHECKING

Understanding the sigmoid and probability output.

MODEL ANSWER

- It runs a linear regression on the log-odds, then squashes the result through the sigmoid into a probability between 0 and 1.
- A threshold (default 0.5) turns the probability into a class.
- Fast, explainable baseline — always worth trying first.

PYTHON

```
sigmoid = lambda z: 1 / (1 + np.exp(-z))
for z in (-4, 0, 2, 4):
    print(f"z={z:>2} -> p={sigmoid(z):.3f}")
```

OUTPUT ✓ REAL RUN

```
z=-4 -> p=0.018
z= 0 -> p=0.500
z= 2 -> p=0.881
z= 4 -> p=0.982
```

Common mistake Saying logistic regression outputs a class directly — it outputs a probability.

Interviewer may ask next How would you use it for 3+ classes? (Multinomial / one-vs-rest.)

12 How does a decision tree decide where to split?

Algorithms Medium

WHAT THE INTERVIEWER IS CHECKING

Impurity (Gini / entropy) in plain words.

MODEL ANSWER

- At each node it tries features and thresholds and picks the split that makes the child groups 'purest'.
- Purity is measured with Gini impurity or entropy: 0 = all one class.
- Pros: easy to explain, no scaling needed. Cons: overfits easily — limit max_depth / min_samples_leaf.

PYTHON

```
def gini(labels):
    p = np.bincount(labels) / len(labels)
    return round(1 - np.sum(p ** 2), 3)

print("mixed  [0,0,1,1]:", gini(np.array([0, 0, 1, 1])))
print("pure   [1,1,1,1]:", gini(np.array([1, 1, 1, 1])))
print("mostly [0,1,1,1]:", gini(np.array([0, 1, 1, 1])))
```

OUTPUT ✓ REAL RUN

```
mixed  [0,0,1,1]: 0.5
pure   [1,1,1,1]: 0.0
mostly [0,1,1,1]: 0.375
```

Common mistake Saying trees need feature scaling — they don't.

Interviewer may ask next What does 'information gain' mean? (Parent impurity minus weighted child impurity.)

13 Random forest vs gradient boosting? Bagging vs boosting?

Algorithms Medium

WHAT THE INTERVIEWER IS CHECKING

The two big ensemble ideas.

MODEL ANSWER

- Bagging (random forest): many deep trees trained in parallel on random samples and random features; average their votes → lower variance.
- Boosting (gradient boosting, XGBoost, LightGBM): shallow trees built one after another, each fixing the previous errors → lower bias.
- Boosting usually wins on tabular data but needs more tuning; random forest is a strong, low-effort baseline.

PYTHON

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import cross_val_score
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
X, y = load_breast_cancer(return_X_y=True)
for name, m in [("Single tree", DecisionTreeClassifier(random_state=0)),
                ("Random forest", RandomForestClassifier(n_estimators=200, random_state=0)),
                ("Grad. boosting", GradientBoostingClassifier(random_state=0))]:
    print(f"{name:15} {cross_val_score(m, X, y, cv=5).mean():.3f}")
```

OUTPUT ✓ REAL RUN

```
Single tree     0.917
Random forest   0.960
Grad. boosting  0.963
```

Common mistake Saying random forest trees are trained one after another — that's boosting.

Interviewer may ask next Why does a random forest also pick random features at each split? (To make trees less correlated.)

14 How does K-means work, and how do you choose K?

Algorithms Medium

WHAT THE INTERVIEWER IS CHECKING

The algorithm steps and the elbow idea.

MODEL ANSWER

- 1. Pick K starting centres. 2. Assign each point to its nearest centre. 3. Move each centre to the mean of its points. Repeat until stable.
- Choose K with the elbow method (inertia vs K) or silhouette score — and whether the segments make business sense.
- Scale features first; K-means uses distances and assumes roughly round clusters.

PYTHON

```
from sklearn.datasets import make_blobs
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
X, _ = make_blobs(n_samples=300, centers=3, random_state=7)
for k in (2, 3, 4, 5):
    km = KMeans(n_clusters=k, n_init=10, random_state=0).fit(X)
    print(f"K={k}  inertia={km.inertia_:8.0f}  silhouette={silhouette_score(X,
km.labels_):.3f}")
```

OUTPUT ✓ REAL RUN

```
K=2  inertia=    3303  silhouette=0.745
K=3  inertia=     573  silhouette=0.790
K=4  inertia=     501  silhouette=0.647
K=5  inertia=     435  silhouette=0.471
```

Common mistake Running K-means on unscaled features like income (lakhs) and age (years).

Interviewer may ask next Name a clustering method that doesn't need K up front. (DBSCAN, hierarchical clustering.)

Features & data preparation

15 When do you need feature scaling? Standardisation vs normalisation?

Features Easy

WHAT THE INTERVIEWER IS CHECKING

Knowing which models care about scale.

MODEL ANSWER

- Needed for distance- or gradient-based models: KNN, K-means, SVM, linear/logistic regression with regularisation, neural networks.
- Not needed for tree-based models.
- Standardisation: (x – mean) / SD → mean 0, SD 1. Min-max normalisation: squeeze into 0–1.
- Fit the scaler on the training data only, then transform the test data.

PYTHON

```
from sklearn.preprocessing import StandardScaler, MinMaxScaler
X_train = np.array([[25, 300000], [30, 600000], [35, 900000]])   # age, income
X_test  = np.array([[28, 450000]])
sc = StandardScaler().fit(X_train)          # learn mean/SD from TRAIN only
print(sc.transform(X_test).round(2))
print(MinMaxScaler().fit(X_train).transform(X_test).round(2))
```

OUTPUT ✓ REAL RUN

```
[[-0.49 -0.61]]
[[0.3  0.25]]
```

Common mistake Scaling the whole dataset before splitting → leakage.

Interviewer may ask next Which scaler handles outliers better? (RobustScaler — uses median and IQR.)

16 How do you encode categorical features?

Features Easy

WHAT THE INTERVIEWER IS CHECKING

One-hot vs label/ordinal and when each is right.

MODEL ANSWER

- One-hot: one 0/1 column per category — for categories with no order (city).
- Ordinal/label encoding: map to numbers — only when there's a real order (Low < Medium < High) or for tree models.
- Many categories (thousands of pin codes)? Group rare ones, or use target/frequency encoding carefully.

```python
df = pd.DataFrame({"city": ["Pune", "Delhi", "Pune"], "size": ["Low", "High", "Medium"]})
print(pd.get_dummies(df["city"], prefix="city", dtype=int))
print(df["size"].map({"Low": 0, "Medium": 1, "High": 2}).tolist())
```

OUTPUT ✓ REAL RUN

```
   city_Delhi  city_Pune
0           0          1
1           1          0
2           0          1
[0, 2, 1]
```

Common mistake Label-encoding cities as 0, 1, 2 for a linear model — it invents an order (Delhi < Pune).

Interviewer may ask next What happens if the test data has a city never seen in training? (OneHotEncoder(handle_unknown='ignore').)

17 How do you handle an imbalanced dataset?

Data preparation · Medium

WHAT THE INTERVIEWER IS CHECKING

Practical options, and the right metrics.

MODEL ANSWER

- First, switch metrics: precision, recall, F1, PR-AUC instead of accuracy.
- Use class_weight='balanced' so mistakes on the rare class cost more.
- Resample: undersample the majority or oversample the minority (e.g. SMOTE) — on the training set only.
- Tune the decision threshold for the business goal.

PYTHON

```
from sklearn.datasets import make_classification
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import recall_score
X, y = make_classification(n_samples=4000, weights=[0.97], flip_y=0, class_sep=0.8,
random_state=3)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, stratify=y, random_state=3)
for cw in (None, "balanced"):
    m = LogisticRegression(class_weight=cw, max_iter=1000).fit(X_tr, y_tr)
    print(f"class_weight={str(cw):8}  minority recall={recall_score(y_te,
m.predict(X_te)):.2f}")
```

OUTPUT ✓ REAL RUN

```
class_weight=None      minority recall=0.10
class_weight=balanced  minority recall=0.90
```

Common mistake Oversampling before the train/test split — copies of the same rows end up in both.

Interviewer may ask next What does SMOTE do? (Creates synthetic minority examples between existing neighbours.)

18 What is regularisation? L1 vs L2?

Overfitting Medium

WHAT THE INTERVIEWER IS CHECKING

How penalties control model complexity.

MODEL ANSWER

- Regularisation adds a penalty for large coefficients so the model can't fit noise.
- L2 (Ridge): shrinks all coefficients towards zero smoothly.
- L1 (Lasso): can push some coefficients exactly to zero → built-in feature selection.
- Strength is a hyperparameter (alpha, or C = 1/strength in scikit-learn classifiers).

PYTHON

```
from sklearn.datasets import load_diabetes
from sklearn.linear_model import Ridge, Lasso
X, y = load_diabetes(return_X_y=True)
ridge = Ridge(alpha=1.0).fit(X, y)
lasso = Lasso(alpha=1.0).fit(X, y)
print("Ridge zero coefs:", int(np.sum(ridge.coef_ == 0)), "of", X.shape[1])
print("Lasso zero coefs:", int(np.sum(lasso.coef_ == 0)), "of", X.shape[1])
```

OUTPUT ✓ REAL RUN

```
Ridge zero coefs: 0 of 10
Lasso zero coefs: 7 of 10
```

Common mistake Forgetting to scale features before regularising — the penalty then depends on units.

Interviewer may ask next What is Elastic Net? (A mix of L1 and L2.)

Putting it together

19 How do you explain which features matter to a model?

Explainability Medium

WHAT THE INTERVIEWER IS CHECKING

Being able to explain a model to a business audience.

MODEL ANSWER

- Linear models: look at coefficients (on scaled features).
- Tree models: built-in feature importance — quick but biased towards high-cardinality features.
- Permutation importance: shuffle one feature and measure how much the score drops — model-agnostic and more reliable.
- SHAP values explain individual predictions as well.

PYTHON

```
from sklearn.datasets import load_breast_cancer
from sklearn.ensemble import RandomForestClassifier
from sklearn.inspection import permutation_importance
data = load_breast_cancer()
X_tr, X_te, y_tr, y_te = train_test_split(data.data, data.target, random_state=0)
rf = RandomForestClassifier(n_estimators=200, random_state=0).fit(X_tr, y_tr)
imp = permutation_importance(rf, X_te, y_te, n_repeats=10, random_state=0)
top = pd.Series(imp.importances_mean, index=data.feature_names).nlargest(3)
print(top.round(3))
```

OUTPUT ✓ REAL RUN

```
worst area         0.021
worst radius       0.020
worst perimeter    0.013
dtype: float64
```

Common mistake Saying 'feature importance shows X causes Y'.

Interviewer may ask next Why can two highly correlated features both look unimportant in permutation importance?

20 Walk me through an ML project end to end.

Workflow Medium

WHAT THE INTERVIEWER IS CHECKING

Structured thinking — this is often the final question.

MODEL ANSWER

- 1. Business goal and success metric. 2. Collect data and explore (EDA). 3. Clean, split, build features.
- 4. Baseline model, then better models, tuned with cross-validation. 5. Evaluate on the untouched test set.
- 6. Explain results, deploy, monitor for data drift, retrain when needed.
- A scikit-learn Pipeline keeps preprocessing and model together, which prevents leakage.

PYTHON

```
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.linear_model import LogisticRegression
df = pd.DataFrame({
    "city":   ["Pune", "Delhi", "Pune", "Mumbai", "Delhi", "Pune", "Mumbai", "Delhi"],
    "visits": [3, 10, 1, 7, 12, 2, 9, 8],
    "spend":  [500, 4000, 200, 2500, 5200, 300, 3000, 3500],
    "churn":  [1, 0, 1, 0, 0, 1, 0, 0],
})
prep = ColumnTransformer([
    ("num", StandardScaler(), ["visits", "spend"]),
    ("cat", OneHotEncoder(handle_unknown="ignore"), ["city"]),
])
pipe = Pipeline([("prep", prep), ("model", LogisticRegression())])
pipe.fit(df.drop(columns="churn"), df["churn"])
new = pd.DataFrame({"city": ["Chennai"], "visits": [2], "spend": [400]})
print("churn probability:", pipe.predict_proba(new)[0, 1].round(2))
```

OUTPUT ✓ REAL RUN

```
churn probability: 0.74
```

Common mistake Jumping straight to 'I'd use XGBoost' without defining the goal or metric.

Interviewer may ask next After deployment, accuracy drops. What could be happening? (Data drift, changed user behaviour, a broken input pipeline.)

BEFORE YOUR INTERVIEW

Revision checklist

Tick a question only when you can answer it out loud in under two minutes and give one real example.

- [ ] 01 What are supervised, unsupervised and reinforcement learning?
- [ ] 02 Classification vs regression — how do you decide?
- [ ] 03 Why do we split into train and test sets? What is data leakage?
- [ ] 04 What is overfitting vs underfitting? How do you spot and fix it?
- [ ] 05 What is k-fold cross-validation and why use it?
- [ ] 06 Why can accuracy be misleading? Explain the confusion matrix.
- [ ] 07 Precision vs recall — when do you care more about each?
- [ ] 08 What is ROC-AUC?
- [ ] 09 MAE vs RMSE vs R^2 for regression?
- [ ] 10 How does linear regression work?
- [ ] 11 Logistic regression is used for classification — why is it called regression?
- [ ] 12 How does a decision tree decide where to split?
- [ ] 13 Random forest vs gradient boosting? Bagging vs boosting?
- [ ] 14 How does K-means work, and how do you choose K?
- [ ] 15 When do you need feature scaling? Standardisation vs normalisation?
- [ ] 16 How do you encode categorical features?
- [ ] 17 How do you handle an imbalanced dataset?
- [ ] 18 What is regularisation? L1 vs L2?
- [ ] 19 How do you explain which features matter to a model?
- [ ] 20 Walk me through an ML project end to end.

Think out loud

Say your approach before you write anything. Interviewers score your reasoning, not just the final answer.

Connect it to the business

End answers with what the number means: “so we should focus on Bengaluru”, “so the model is safe to launch”.

Know your own projects

Expect “where did you use this?” after almost every question. Keep one project story ready per topic.

Say what you'd check

Mention nulls, duplicates, leakage or edge cases before being asked — it signals real experience.

Prepared the questions?
Now land the interview.

FirstOffer lists fresh fresher jobs, internships and off-campus drives every day — with direct HR emails, Google Forms and official apply links. No middlemen.

firstoffer.online

Daily job alerts on our WhatsApp Channel · Questions vary by company and role; understand the concepts, don't just memorise answers.